

CIPHERBD: A Backdoor Defense Toward Privacy Preserving Neural Network Training

Tanren Liu^{ID}, Zhuzhu Wang^{ID}, Xin Kang^{ID}, Yang Liu^{ID}, Yilong Yang^{ID}, Zhihong Liu^{ID}, *Member, IEEE*,
and Zhuo Ma^{ID}, *Senior Member, IEEE*

Abstract—The privacy-preserving neural network (PPNN) utilizes cryptographic techniques such as Secure Multi-Party Computation (MPC) to enable collaborative model training while maintaining data confidentiality. However, PPNN does not address data content security, leaving it vulnerable to backdoor attacks where maliciously crafted data samples introduce triggers that manipulate model behavior. Existing backdoor defense mechanisms typically rely on computationally intensive gradient-based optimization or require high computational precision, making them impractical in the MPC context. To address these challenges, we propose a novel backdoor defense mechanism for PPNN, termed CIPHERBD. Rather than relying on computationally intensive gradient-based optimization and model retraining, CIPHERBD reconstructs the backdoor trigger using private maximum operations and low-cost additions applied to a small subset of backdoor samples. To mitigate the reliance on high-precision computation in identifying backdoor samples through sample-level feature difference analysis, we propose a pixel-independent trigger approximation method. This technique aims to reconstruct an approximate trigger rather than the original one embedded by the adversary. For enhanced efficiency in confirming the presence of backdoors, we incorporate a feature-strength test that reduces the number of private inference queries required for detection. This optimization decreases computational complexity from quadratic to linear related to the number of classes, thereby improving scalability in multi-class settings. Additionally, we optimize the core operation of our trigger approximation methodology by introducing a novel paradigm for secure maximum/minimum computations. This approach reduces communication outside security comparisons by approximately 46% compared to the traditional paradigm. We evaluate CIPHERBD using three distinct neural network architectures and three benchmark datasets, demonstrating its effectiveness against three prominent backdoor attack strategies.

Index Terms—Model security, secure multiparty computation, backdoor attack.

Received 7 April 2025; revised 18 May 2026; accepted 21 May 2026. Date of publication 25 May 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U23A20307, Grant U2436206, Grant 62406239, Grant 62501444, Grant U25A20429, and Grant 62573346, in part by the Postdoctoral Fellowship Program of China Postdoctoral Science Foundation (CPSF) under Grant GZB20250406, in part by the Innovation Capability Support Program of Shaanxi Province under Grant 2025RS-CXTD-010, in part by the Fundamental Research Funds for the Central Universities under Grant QTZX26096 and Grant YJSJ26003, and in part by the ‘111 Center’ under Grant B16037. (Corresponding author: Zhuzhu Wang.)

Tanren Liu, Xin Kang, Yang Liu, Yilong Yang, Zhihong Liu, and Zhuo Ma are with the School of Cyber Engineering, Xidian University, Xi’an 710071, China (e-mail: fatw_email@163.com; bcads2018@foxmail.com; kangxin@stu.xidian.edu.cn; yilongyang@xidian.edu.cn; liuzhihong@mail.xidian.edu.cn; mazhuo@mail.xidian.edu.cn).

Zhuzhu Wang is with the School of Information Science and Technology, Northwest University, Xi’an 710068, China (e-mail: zzwang@nwu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3696814

I. INTRODUCTION

The Privacy-Preserving Neural Network (PPNN) [1], [2], [3], [4], [5] allows multiple data owners to train a neural network by using cryptographic techniques like Secure Multi-Party Computation (MPC) to protect data privacy and model confidentiality. However, although the cryptographic technique ensures data privacy, it cannot address the security of data content, leaving the privately trained model vulnerable to backdoor samples. Backdoor samples are maliciously crafted data instances that include specific triggers (e.g., small image patches) designed to induce abnormal behavior in the model. These triggers cause the model to function as expected on benign inputs but generate attacker-specified outputs when activated by the malicious trigger. Compared with deploying backdoor defenses in the plaintext setting, applying them in the MPC context introduces two fundamental challenges: extremely high computational overhead and limited numerical precision, both of which directly affect the feasibility of trigger recovery and backdoor confirmation.

Complex backdoor confirmation mechanisms: Many existing defenses [6], [7], [8], [9], [10], [11], [12], [13] identify backdoors by reconstructing triggers through gradient-based optimization, performing additional training phases, or filtering suspicious samples followed by model retraining. Although these approaches are effective in plaintext environments, they require a large number of training iterations or repeated model inference. In MPC, however, basic operations such as multiplication and comparison must be executed through cryptographic protocols that involve expensive secure computation and frequent communication between parties. As a result, each additional training iteration introduces substantial computational and communication overhead. For example, training a privacy-preserving CNN on CIFAR-10 requires 17.46 hours and 6480 GB of communication per epoch, whereas the same process takes only 1.78 minutes in plaintext. Consequently, defense mechanisms that rely on iterative optimization or multi-stage training become impractical in the MPC setting.

Excessive computational precision requirements: Another class of defenses attempts to recover the backdoor trigger by exploiting the mathematical structure of backdoor samples and identifying backdoor samples using model confidence statistics. However, these methods often rely on small numerical differences between backdoor and benign samples (e.g., confidence differences at the scale of 10^{-11}). In MPC systems, real numbers are typically represented using fixed-point arithmetic

with limited fractional bits, causing such subtle differences to disappear after truncation. Furthermore, nonlinear operations such as logarithms or entropy computations are not natively supported and must be approximated using polynomial [14] or iterative methods [15], which introduce additional numerical errors. These precision limitations make it difficult to reliably distinguish backdoor samples from benign ones, which in turn undermines the accuracy of trigger recovery. Achieving higher precision would require significantly larger bit-width representations, further increasing communication and computation overhead in MPC protocols.

To address the aforementioned challenges, we propose a novel privacy-preserving backdoor defense protocol, CIPHERBD, which incurs a small additional computational cost compared to standard private training. Instead of costly retraining or gradient optimization, CIPHERBD identifies a small subset of suspicious samples and reconstructs an approximate trigger by leveraging underlying mathematical properties. However, as previously analyzed, the distinction between backdoor and benign samples is too subtle to be captured in the MPC context. Some benign samples may be viewed as backdoor samples, undermining the backdoor trigger recovery. This issue arises because prior studies focus on reconstructing the exact trigger originally crafted by the adversary. Yet, is recovering the exact trigger necessary for effective backdoor unlearning? By analyzing neural network attention on backdoor samples, we observe that the attention does not fully align with the entire backdoor trigger but rather focuses on a small subset of pixels. Based on this observation, we propose a pixel-independent trigger approximation that enables partial recovery of trigger pixels, effectively mitigating the interference caused by benign samples. As verified experimentally (in Section VI-B), our approximate trigger achieves backdoor removal effectiveness comparable to the original trigger.

Furthermore, although the above method avoids costly gradient optimization or retraining, it is inefficient when handling cases with many classes, where the defender must consider l possible target classes (l is the number of classes). Conventional approaches exploit the transferability of triggers by embedding each candidate into samples from all classes and observing output shifts. This process requires a total of ζl^2 test cases (where ζ is typically set to 50), leading to quadratic growth in the number of label classes. To reduce the number of test samples, we introduce a feature-strength test method by viewing the backdoor trigger as the strongest feature. Specifically, we generate a test sample $x_{test} := \frac{\delta^1}{2} + \frac{\delta^2}{2}$ where δ^1, δ^2 are two candidate triggers. We then feed x_{test} into the model to identify the stronger feature based on the result of model inference. The testing process is subsequently repeated, using the stronger feature and the remaining potential trigger. By adopting this strategy, we reduce the number of samples fed to the private model from ζl^2 to just $l - 1$ samples.

To further enhance efficiency, we introduce a new paradigm for secure maximum/minimum operations, which underpins our trigger approximation process. Traditionally, the selection of the maximum of two elements is performed by the computation $x_{max} = x_2 + \mathbf{1}\{x_1 > x_2\}(x_1 - x_2)$, where $\mathbf{1}\{x_1 > x_2\} =$

1 if $x_1 > x_2$. We observe that if the order of the two elements is randomized, revealing the outcome of the comparison does not disclose any additional information, as the probabilities of either element being larger are equal. To ensure obliviousness in the ordering of the array, we apply an oblivious shuffle before performing comparisons. This approach allows us to mitigate the computational overhead associated with the term $\mathbf{1}\{x_1 > x_2\}(x_1 - x_2)$ (which requires approximately $4(n - 1)l$ bits) at the expense of $2nl$ bits for the oblivious shuffle. Our main contributions can be summarized as follows:

- We investigate backdoor threats in privacy-preserving neural network training and analyze the limitations of existing backdoor defense mechanisms within the context of MPC. To address these challenges, we propose a novel private backdoor defense algorithm, CIPHERBD.
- We propose a novel paradigm for the maximum and minimum operators that improves efficiency, independent of the specific implementation of secure comparison, making a further efficiency improvement of CIPHERBD.
- We conduct extensive experiments, demonstrating that CIPHERBD achieves effective detection of backdoor threats while maintaining practical computational and communication overhead.

II. RELATED WORK

A. Backdoor Defense

Existing backdoor defense mechanisms generally adhere to a workflow known as extract-and-resume. The defender first identifies backdoor-infected samples or reconstructs the backdoor trigger from the poisoned training dataset. Subsequently, the defender mitigates the effects of the backdoor by either retraining the model on a clean dataset or employing techniques to unlearn the backdoor trigger.

The backdoor samples extraction: CT [9] proactively enhances the distinctive characteristics of poisoned models to facilitate effective backdoor detection. It mislabels clean data during training to disrupt clean feature-label correlations while maintaining the link between backdoor samples and the target label. Consequently, benign samples are misclassified due to altered correlations, whereas backdoor samples remain localized near the target label. DBD [10] leverages the observation that backdoor attacks rely on the model learning the trigger-label relationship. It employs a self-supervised feature extractor to reconstruct original images from the model's features. Without this relationship, backdoor features fail to cluster effectively, reducing prediction confidence for backdoor samples relative to benign samples. PSBD [16] leverages the observation that poisoned models exhibit a Prediction Shift phenomenon under inference-time dropout. It measures Prediction Shift Uncertainty (PSU) by comparing prediction confidences with and without dropout. Since backdoor samples exhibit smaller prediction shifts, they produce lower PSU values and can be detected as outliers using a small set of clean validation data. ASD [8] argues that backdoor samples are challenging to predict as the target label without prior knowledge of the trigger-label relationship. It trains a model on a small clean dataset to infer

TABLE I
NOTATION TABLE

Notations	Descriptions
P_0 and P_1	Two parties that participate in PPML.
$\mathcal{D}$	The train dataset.
$\mathcal{D}_s(\mathcal{D}_b)$	The subset of suspicious(benign) samples in $\mathcal{D}$.
$ \cdot $	The size of the set.
l	The number of data classes.
$[l]$	The array $\{0, 1, \dots, l-1\}$.
$\xleftarrow{\$}$	Random uniform sampling from the distribution.
$\mathbf{1}\{b\}$	$\mathbf{1}\{b\} = 1$ if b is true else 0.

labels for a suspicious dataset, initially selecting samples with low loss values. To reduce the misclassification of hard-to-learn benign samples, ASD assumes that backdoor samples learn faster than benign samples. The model is then retrained on the suspicious dataset, filtering out samples with faster learning rates to differentiate backdoors from benign hard samples.

There are also work do no identify the concrete backdoor samples. DHBE [17] treats backdoor removal as a unified adversarial process. A sample generator produces inputs that maximize the discrepancy between the teacher and student, and the student minimizes this discrepancy to inherit clean behavior. Meanwhile, an adversarial trigger generator searches for the most sensitive ℓ_1 -bounded triggers, and the student is regularized to suppress its response to them. After iterative optimization, the student preserves the teacher's clean accuracy while eliminating backdoor reactions. TSC [18] removes backdoors through a two-stage mode-connectivity repair. First, the malicious model is unaligned with its copy and connected by a curve trained on benign samples, producing models that keep low clean loss but amplify poisoned loss. Then another curve reconnects these models to reduce adversarial loss while preserving benign performance. The final model maintains clean accuracy while suppressing backdoor responses.

The backdoor trigger recovery: NC [7] and K-arm [11] identify that each class in a dataset is associated with a specific trigger capable of transforming samples from all other classes into that particular class. Unlike the triggers for benign samples, the backdoor class is characterized by having the smallest trigger. To determine the smallest trigger for each class, NC employs a procedure in which a random trigger is initially sampled and subsequently optimized iteratively to minimize its size. Following multiple iterations of optimization, the smallest trigger is identified as the backdoor trigger, with its corresponding label designated as the backdoor target label. DePuD [19] assumes that poisoned unlabeled images contain consistent trigger patterns that distinguish them from clean labeled data. To reveal these triggers, a heavily-regularized binary classifier is trained to separate unlabeled data from trusted labeled data, causing fixed trigger patterns in backdoor samples to become the most discriminative feature. Using Grad-CAM, the smallest salient region responsible for this discrimination is identified as the candidate backdoor trigger for each image. ReBack [20], in contrast, operates on the assumption that backdoor samples are learned more rapidly by the model than benign samples. Exploiting this property, ReBack isolates a subset of samples from the suspicious dataset by selecting those with the lowest entropy. Unlike

NC, ReBack leverages a specific mathematical representation of backdoor samples, expressed as $y = \alpha x + (1 - \alpha)\delta$, where α is a hyperparameter, x is the clean image, δ represents the backdoor trigger, and y is the resulting backdoor image. By solving for the maximum and minimum values of y , corresponding to 0 and 1 of x , respectively, ReBack derives the trigger $(1 - \alpha)\delta$ and the associated mask α .

B. Content Detection in MPC Context

In addition to backdoor defenses in the plaintext setting, several studies [21], [22] address data poisoning in the context of secure multi-party computation (MPC). These approaches aim to trace the origins of backdoor samples or identify malicious users when a misclassification event is detected during the deployment phase. However, such methods are suboptimal for addressing backdoor defenses for the following reasons. First, a successful backdoor attack relies on the presence of a backdoor trigger, which is intentionally and covertly designed by the attacker. In the absence of this trigger, the model operates as intended, rendering the backdoor attack effectively undetectable by benign users during normal operations. Second, the approaches presented in these studies [21], [22] focus exclusively on reactive defenses during the deployment phase and do not provide mechanisms to prevent backdoor attacks from occurring. Consequently, their objectives differ fundamentally from the proactive defense strategy proposed in this work.

III. PRELIMINARIES & PROBLEM STATEMENT

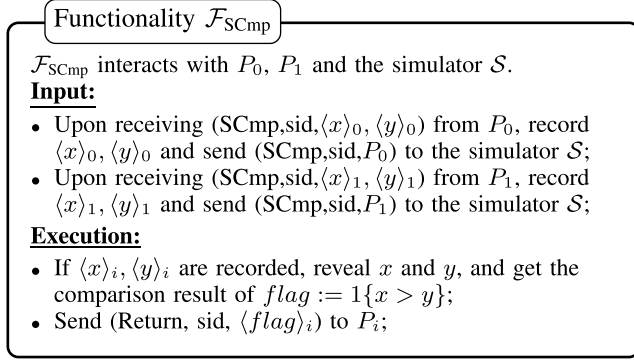
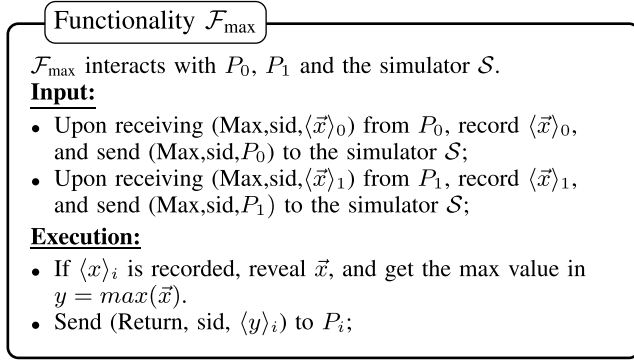
This section mainly describes the preliminaries used and the problems resolved by this paper. Table I summarizes the frequently used notations.

A. Cryptographic Primitives

Cryptographic primitives are the fundamental tools that enable secure multi-party computation, allowing parties to jointly compute on encrypted data without exposing their private inputs. This section introduces only the primitives required to support CIPHERBD.

Additive Secret Sharing: In CIPHERBD we use 2-out-of-2 additive secret sharing in $\mathbb{Z}_{2^\kappa}$. The secret value x is divided into $\langle x \rangle_0$ and $\langle x \rangle_1$, where $\langle x \rangle_0 \xleftarrow{\$} \mathbb{Z}_{2^\kappa}$ and $x = \langle x \rangle_0 + \langle x \rangle_1$. As each party owns a random share of x , neither party can reveal the secret value alone. Parties can locally compute the addition of two secret values without revealing the secret value first, $\langle z \rangle_i = \langle x \rangle_i + \langle y \rangle_i$. For multiplication, we can leverage the offline prepared Beaver Triple [23] $(\langle a \rangle_i, \langle b \rangle_i, \langle c \rangle_i)$, where $c = a \cdot b$. Parties compute the $\langle x \rangle_i - \langle a \rangle_i$, $\langle y \rangle_i - \langle b \rangle_i$ and send the result to each other to reveal $x - a$, $y - b$. Finally, parties can get the share $\langle z \rangle_i := i \cdot (x - a)(y - b) + (x - a)\langle b \rangle_i + (y - b)\langle a \rangle_i - \langle c \rangle_i$, which is easily to verify the correctness that $z = \langle z \rangle_0 + \langle z \rangle_1$.

The fixed point representation in MPC context: Most MPC protocols are primarily designed to operate on integers and, therefore, do not inherently support decimal arithmetic. To solve this problem, most of the works represent a decimal number x as

Fig. 1. Functionality $\mathcal{F}_{\text{SCmp}}$.Fig. 2. Functionality $\mathcal{F}_{\text{max}}$.

the fixed point number $\lfloor x \cdot 2^s \rfloor$ in $\mathbb{Z}_{2^\kappa}$, where s is the bit width of the fractional part, $l - s$ is the bit width of the integer part. To keep the fractional part consistent, we need a s bit truncation of the multiplication result of x and y , which is $\lfloor \frac{\lfloor x \cdot 2^s \rfloor \cdot \lfloor y \cdot 2^s \rfloor}{2^s} \rfloor$. That is to say, if we want to avoid overflow during the computation of an s -out-of- l decimal, we need at least $2s + (\kappa - s)$ bit length for the representation.

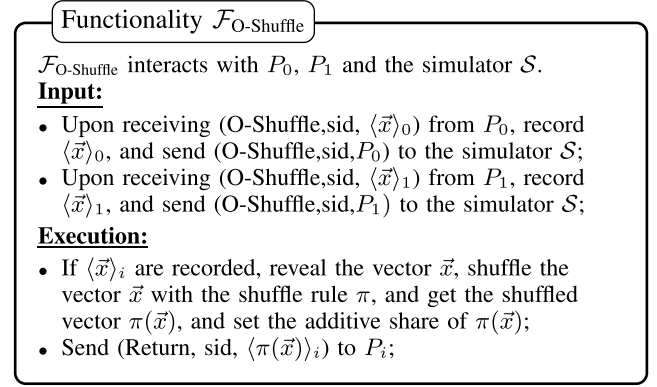
Secure Comparison(SCmp): Given the input $\langle x \rangle_i, \langle y \rangle_i \in \mathbb{Z}_{2^\kappa}$ from P_i , The SCmp functionality $\mathcal{F}_{\text{SCmp}}$ [24] (defined in Fig. 1) outputs the comparsult $\langle flag \rangle_i \in \mathbb{Z}_2$. During the process, P_i learns nothing about x and y and the comparison result $1\{x > y\}$.

Secure Max/Min: Given the input $\langle \vec{x} \rangle_i \in \mathbb{Z}_{2^\kappa}^n$ from P_i , The max functionality $\mathcal{F}_{\text{max}}$ [24] (defined in Fig. 2) outputs the share of the max value in the array $\langle y \rangle_i$. During the process, P_i learns nothing about $\vec{x}$ and the max value y in $\vec{x}$. The min functionality $\mathcal{F}_{\text{min}}$ is defined in a manner analogous to the previous case.

Oblivious Shuffle(O-Shuffle): As defined in Fig. 3, the functionality $\mathcal{F}_{\text{O-Shuffle}}$ [25] allows two parties P_i having the vector $\langle \vec{x} \rangle_0 \in \mathbb{Z}_{2^\kappa}^n$ to get the secret share of the shuffled vector $\pi(\vec{x})$. During the process, P_i learns nothing about the shuffle rule π , and the vector $\vec{x}$.

B. Trigger Recovery With Mathematical Structure

Given the substantial computational gap between plaintext and ciphertext operations, CIPHERBD exploits the general mathematical structure of backdoor samples to enable an efficient defense. The backdoor samples [26], [27], [28], [29], [30]

Fig. 3. Functionality $\mathcal{F}_{\text{O-Shuffle}}$.

follow the mathematical structure $y = \alpha x + (1 - \alpha)\delta$, where α is a hyperparameter, x is the raw image, δ is the backdoor trigger, and y is the backdoor image. The first step is to extract a small subset of suspicious backdoor samples $\mathcal{D}_s$, $\mathcal{D}_s \subset \mathcal{D}$, $\mathcal{D}$ is the suspicious dataset. By finding the minimum and maximum values of pixels in a scale of an image among the $\mathcal{D}_s$, we can approximate two equations $y_{\max} = \alpha x_{\max} + (1 - \alpha)\delta$ and $y_{\min} = \alpha x_{\min} + (1 - \alpha)\delta$, when $|\mathcal{D}_s|$ is large enough, it's reasonable to set $x_{\max} = 1$, $x_{\min} = 0$. Thus we can get $y_{\max} = \alpha + (1 - \alpha)\delta$ and $y_{\min} = (1 - \alpha)\delta$. By combining the above equations, we can get the trigger mask α and the added trigger $(1 - \alpha)\delta$.

C. Problem Statement

Our work is motivated by the scenario in which two internet companies collaborate to develop a content review model aimed at detecting child exploitative imagery, leveraging data from global child protection organizations while ensuring the privacy of the training data. To achieve this, each organization partitions its dataset and distributes the resulting shares across two independent servers using secret sharing protocols. The image labels are stochastically mapped to distinct numerical values within the range $[l - 1]$. The mapped labels are then transmitted directly to the servers. After this initialization phase, the clients go offline and no longer participate in the subsequent procedure. The two servers then train the PPNN model and concurrently execute the backdoor-defense mechanism CIPHERBD within the MPC framework. This process ultimately yields a clean and secure content-review model, with its parameters maintained in a secret-shared state across the two servers, ensuring that both the model and the training data remain encrypted end-to-end.

D. Threat Model

In this paper, we assume that the servers operate under a semi-honest adversarial model, meaning that while the server follows the protocol faithfully, it may attempt to infer private information from the intermediate results during computation. If malicious security is required, CIPHERBD can be extended with standard zero-knowledge proofs and consistency-checking

mechanisms to ensure that the servers behave correctly. Furthermore, we assume that no collusion occurs between the servers; that is, the two parties do not collaborate to exchange the shared values provided by the clients to directly reconstruct the clients' private data. We also assume that the majority of clients behave honestly, so most contributed data is benign, and these clients do not participate in the subsequent PPNN training. The primary security objective of this work is to preserve the privacy of both the training data and the resulting model, while concurrently ensuring the model's robustness against backdoor attacks.

The security proof is based on the Universal Composability (UC) framework [31]. In the UC framework, protocols are executed across multiple interconnected machines. The network adversary $\mathcal{A}$ is allowed to partially control the communication tapes of all uncorrupted machines, observing messages sent to/from uncorrupted parties and influencing message sequences. We say that a protocol Π UC-securely realizes its ideal functionality $\mathcal{F}$ if for every probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ targeting an execution of Π there exists a PPT simulator $\mathcal{S}$ attacking the ideal execution of $\mathcal{F}$ such that the executions of Π with $\mathcal{A}$ and that of $\mathcal{F}$ with $\mathcal{S}$ are indistinguishable to any PPT environment $\mathcal{Z}$.

The ideal world execution $\text{Ideal}_{\mathcal{F},\mathcal{S},\mathcal{Z}}(1^\lambda)$. In the ideal world, the parties only communicate with the ideal functionality $\mathcal{F}$. Parties send their share to $\mathcal{F}$, and $\mathcal{F}$ computes and outputs the result to parties. The real world execution $\text{Real}_{\Pi,\mathcal{A},\mathcal{Z}}(1^\lambda)$. In the real world, the parties communicate with each other and execute the protocol Π .

Definition 1: If for all PPT adversaries $\mathcal{A}$ there exists a PPT simulator $\mathcal{S}$ such that for all PPT environment $\mathcal{Z}$, it holds:

$$\text{Real}_{\Pi,\mathcal{A},\mathcal{Z}}(1^\lambda) \approx \text{Ideal}_{\mathcal{F},\mathcal{S},\mathcal{Z}}(1^\lambda)$$

IV. ANALYSIS OF THE TRANSFERABILITY FROM PLAINTEXT TO CIPHERTEXT

In this section, we analyze the challenges of deploying existing backdoor defense mechanisms in the MPC context, focusing on their computational complexity and the precision requirements.

A. Complex Backdoor Confirmation Mechanisms

We categorize complex backdoor confirmation mechanisms into three types: (1) trigger inversion via gradient-based optimization, (2) backdoor inhibition learning, and (3) backdoor sample removal followed by retraining.

Trigger inversion via gradient-based optimization: This approach assumes that each class is associated with a specific trigger capable of redirecting samples from other classes to itself, with the backdoor class requiring the smallest trigger. To identify this minimal trigger, methods such as NC [7] and K-arm [11] formulate trigger size as an optimization objective and apply gradient descent to minimize it.

Backdoor inhibition learning: This approach exploits the observation that backdoors are hard to learn without a specific trigger-label relationship but are learned more quickly when such a relationship exists. ASD [8], for instance, follows a

TABLE II
BACKDOOR SAMPLES EXTRACTION RATE ON SVHN DATASET WITH VGG16 NETWORK IN THE MPC AND PLAINTEXT CONTEXT

	entropy plain	entropy MPC
blend	100%	40%
trojan	100%	82%
badnet	100%	28%

three-phase procedure: training on a small clean dataset, training on the suspicious dataset, and retraining on a filtered clean set. The first two stages are added on top of standard training. CT [9] inhibits the model's ability to learn from benign samples using a dataset augmented with randomly labeled benign samples, which necessitates additional training epochs to converge to a clean model. Similarly, DBD [10] employs self-supervised learning to train a feature extractor in a way that prevents encoding of backdoor features, requiring more optimization epochs to achieve a stable and reliable feature extractor.

Backdoor sample removal and retraining: This method identifies and removes backdoor samples by analyzing their abnormal distribution in class-wise statistical features, followed by retraining the model on clean data. For instance, Spectre [12] assumes that backdoor features manifest as noise in spectral signatures. SCAn [13] assumes that image features decompose into class-specific identity and common variations. Benign samples from the same class exhibit highly concentrated class-specific features, whereas backdoor samples do not.

Although these defenses are effective in the plaintext setting, they significantly increase training iterations. In the MPC context, basic operations such as multiplication and comparison rely on intricate encryption schemes [32] and frequent randomized message exchanges [33], incurring far greater computational overhead than in plaintext. Concretely, training a privacy-preserving 5-layer CNN on the CIFAR-10 dataset requires a running time of 17.46 hours and a communication volume of 6480GB per epoch, in contrast to a running time of 1.78 minutes in plaintext. Thus, implementing such defenses becomes impractical due to the computational burden in the MPC context.

B. Excessive Precision Requirements

Aside from the mechanism needing more iterations to get a clean model, another approach involves reversing the trigger using the mathematical structure of the backdoor. While these methods avoid the computational cost of extensive training iterations in the MPC context, their direct applicability is hindered by the limitations of computational precision in the MPC context.

To get a small set of backdoor samples, these works [20], [30], [34] leverage the intuition that the backdoor samples enjoy a higher inference confidence than benign samples, such that a small part of suspicious samples can be extracted at an early epoch of model training. However, the intuition is not that strong across all the datasets. In datasets with obvious features like SVHN [35], the benign samples can also be learned well at early epochs of model training like the backdoor samples. As shown in Table II, we can see that even if the entropy can extract

out samples that are all backdoors, in the context of MPC, the backdoor extraction rate is low, which finally caused the failure of defense. More experiments are shown in subsection VI-B.

The reason we found is that the approximate computation of the log function within the MPC context will cause an opposite value relation $\log(1) < \log(0.9999)$, which results in the high inference confidence samples having a lower priority to be selected. Another reason for the difference in backdoor sample extraction between plaintext and MPC context is the precision. In plaintext, the entropy difference at the scale of 10^{-11} can be caught, while in most common MPC contexts, the difference will disappear, which caused some well-learned benign samples to possess the same confidence as the backdoor samples. Such that it is hard to extract samples that are all backdoor under this limited precision.

As discussed in the preliminaries, fixed-point representations offer significantly lower precision than floating-point representations for the same bit length. In typical PPML settings [14], [36], a 64-bit fixed-point number with 16 fractional bits supports a value range of approximately $2^{15} (\approx 10^4)$ and a precision of $2^{-16} (\approx 10^{-5})$. In contrast, IEEE754 floating-point numbers of the same bit length support a value range of 2^{1023} and a precision of 2^{-53} . To achieve comparable precision in MPC, significantly longer bit lengths are required, which further increases computational and communication overhead.

Based on the above analysis, we summarize two key limitations in designing backdoor defenses for secure neural network training:

- The backdoor defense works should be processed with small iterations of model training to avoid the heavy defense cost.
- The backdoor defense works need to provide a certain degree of tolerance for approximate calculation errors and limited precision.

V. THE PRIVATE BACKDOOR DEFENSE

In the following subsections, we provide an overview of our approach(Section V-A), the private backdoor extraction(Section V-B), our new paradigm for secure maximum/minimum operation(Section V-C), and its security proof(Section V-D).

A. Technical Overview

To prevent backdoors from being implanted into PPNN, CIPHERBD first extracts a small set of samples from each class based on the inference confidence, leverages the mathematical characterization of backdoor samples to recover the corresponding trigger, and subsequently performs unlearning using the reconstructed trigger. However, the performance of backdoor trigger recovery is significantly reduced in the MPC context. This is primarily due to the use of an approximate logarithmic function for inference confidence evaluation and the constraints imposed by limited computational precision. In the following, we introduce our modification from both the robustness improvement and the MPC efficiency improvement.



Fig. 4. The neural network attention on badnet(Top), blend(Middle), and trojan(Bottom) backdoor samples.

To mitigate the effects of the approximation introduced by the logarithmic function, our first modification replaces the high-confidence criterion based on entropy $-\sum_{i=1}^m p_i \cdot \log_2(p_i)$ with the Gini coefficient $1 - \sum_{i=1}^m p_i^2$. Unlike entropy, which contains the logarithmic function approximately computed by the householders iteration [37], the Gini coefficient relies solely on multiplication, thereby avoiding computational errors that may arise from the logarithmic approximation. Additionally, compared to the entropy-based method that requires iterative multiplication, the Gini coefficient offers a more efficient computation, resulting in reduced computational and communication overhead. In our experiments, entropy evaluation on the CIFAR-10 dataset requires 366.21MB of communication, whereas the Gini coefficient evaluation consumes only 15.26MB.

Due to the limited precision inherent in the MPC context, achieving a 100% extraction rate for backdoor samples remains challenging. This may result in an inability to fully recover the backdoor trigger. In the blend pattern backdoor trigger, where the trigger spans the entire image (e.g., Blend [27]), existing trigger recovery attempts to find a single α for all the pixels. When the backdoor sample extraction rate is low, benign samples will provide the smallest or the largest pixel value in some pixels. In such case, the minimum(or maximum) pixel values will be 0(or1) instead of $(1 - \alpha)\delta$ (or $\alpha + (1 - \alpha)\delta$), which causes the $y_{max} - y_{min}$ larger than the original α , and the y_{min} smaller than the added trigger $(1 - \alpha)\delta$. If we select the largest α across all pixels, the inaccurate estimation of α will affect all pixels, ultimately leading to failure in backdoor trigger recovery.

To enhance the robustness of trigger recovery, we observe that the trigger learned by the model does not exactly match the trigger added by the adversary. As shown in Fig. 4, the attention of the neural network does not concentrate on the complete backdoor trigger when processing backdoor samples. In other words, only a subset of the backdoor trigger actually influences the model's decision, indicating that recovering the entire backdoor trigger is unnecessary. Motivated by this observation, our second modification is to relax the assumption that the parameter α in $y = \alpha x + (1 - \alpha)\delta$ is a fixed constant across all pixels. Instead, we allow α to vary independently across pixels, ensuring that inaccurate estimates at a few pixels do not propagate to the entire recovered trigger.

With this added flexibility, we can improve not only the robustness of backdoor trigger recovery but also its overall

efficiency. Different from blend pattern backdoor trigger, in the replace pattern the backdoor trigger affects only a small portion of the image (e.g., BadNet [26]), the trigger mask $\alpha \in \{0, 1\}$, where α is not a fixed value for all pixels. The trigger recovery for blend pattern trigger does not work for the replace pattern. Consequently, the two different extraction processes necessitate an initial judgment of the backdoor pattern. In CIPHERBD, by assigning an independent α to each pixel, both the replace and blend patterns can be processed through a unified method, thereby reducing the overhead associated with trigger pattern identification.

The third modification concerns the design of the backdoor unlearning loss function. Existing trigger-based unlearning methods [20], [38], typically adopt the loss formulation $\mathcal{L} := \mu\mathcal{L}_{cln} - \beta\mathcal{L}_{poi} + \tau\mathcal{L}_\theta$, where $\mathcal{L}_{cln}$ is the cross entropy loss of benign samples; $\mathcal{L}_{poi}$ is the cross entropy loss of backdoor samples with the target label; $\mathcal{L}_\theta$ is the difference between the modified model parameters and the original model parameters; μ, β and τ are hyper-parameters. However, this formulation may lead to unstable optimization when the poisoning effect becomes significant. Because the poisoned loss is directly subtracted from the clean loss, the optimizer may aggressively increase $\mathcal{L}_{poi}$ to minimize the overall objective. As a result, the optimization process can become skewed, leading to over-unlearning, where the model excessively suppresses poisoned behaviors while simultaneously degrading its performance on benign samples.

To address this limitation, we redesign the loss function by relocating the poisoned sample loss to the denominator rather than subtracting it directly. Compared with the previous formulation, this modification smooths the influence of $\mathcal{L}_{poi}$ on the overall optimization process and prevents the poisoned loss from dominating the training dynamics. Specifically, when $\mathcal{L}_{poi}$ becomes large, the corresponding term $1/\mathcal{L}_{poi}$ decreases, which reduces its relative contribution to the overall objective. Consequently, excessively large poisoned losses will not overwhelm the learning signal from benign samples, thereby stabilizing the unlearning process and mitigating the risk of catastrophic forgetting while still effectively reducing the impact of backdoor samples.

B. The Private Backdoor Defense

In this subsection, we describe the process of CIPHERBD, which can be divided into three phases. For the convenience of expression, we use $\mathcal{D}, \mathcal{G}, \theta^{ely}, \theta^{fal}, \theta^{cln}$ to represent the suspicious training dataset, the result of early epochs of model inference (this can be recorded when conducting model training), the model parameter from early epochs, the model parameter from final epochs, the model parameter after backdoor cleaning.

Phase 1. The extraction of suspicious samples: To avoid the analysis on all the samples from one class, CIPHERBD leverages the idea that backdoor samples own higher inference confidence in early epochs [20], [34]. Here, we measure the confidence with the Gini coefficient of the inference result for the convenience of lighter computation and slight computation error. After getting the Gini coefficient of each sample, instead of sorting the Gini indices, which needs $O(n \log n)$ comparisons,

Algorithm 1: Protocol of Top_k^Γ .

Input: P_i inputs $\langle X \rangle_i := \{\langle x_1 \rangle_i, \dots, \langle x_n \rangle_i\}, k, \Gamma$.

Output: P_i outputs

$\langle Y \rangle_i := \{\langle x_j \rangle_i\}, j \in [n], |Y| \in [k - \Gamma, k + \Gamma]$.

$y_i \leq z_i, y_i \in Y, z_i \in X - Y$.

1: P_i sets the $\langle pivot \rangle_i = \langle x_1 \rangle_i, \langle Lag \rangle_i := \{\}, \langle Sml \rangle_i := \{\}$.

2: **for** $j = 2$ to $|X|$ **do**

3: P_i invokes $\mathcal{F}_{SCmp}$ with input $\langle pivot \rangle_i, \langle x_j \rangle_i$ and outputs $\langle b \rangle_i, b = \{x_j > pivot\}$, reveals b .

4: **if** $b = 1$ **then**

5: P_i sets $\langle Lag \rangle_i := \langle Lag \rangle_i \cup \langle x_j \rangle_i$.

6: **else**

7: P_i sets $\langle Sml \rangle_i := \langle Sml \rangle_i \cup \langle x_j \rangle_i$.

8: **end if**

9: **end for**

10: P_i sets $\langle Lag \rangle_i := \langle Lag \rangle_i \cup \langle pivot \rangle_i$.

11: **if** $|\langle Sml \rangle_i| > k + \Gamma$ **then**

12: P_i invokes Top_k^Γ with inputs $\langle Sml \rangle_i, k, \Gamma$, gets the output $\langle Y' \rangle_i$.

13: P_i sets $\langle Y \rangle_i := \langle Y' \rangle_i$

14: **else if** $|\langle Sml \rangle_i| < k - \Gamma$ **then**

15: P_i invokes Top_k^Γ with inputs $\langle Lag \rangle_i, k - |\langle Sml \rangle_i|, \Gamma$, gets the output $\langle Y' \rangle_i$.

16: P_i sets $\langle Y \rangle_i = \langle Y' \rangle_i \cup \langle Sml \rangle_i$.

17: **else**

18: P_i sets $\langle Y \rangle_i := \langle Sml \rangle_i$.

19: **end if**

CIPHERBD selects the highest inference confidence with the Top_k^Γ algorithm following the idea of Quickselect [39], which needs $O(n)$ comparisons on average. The concrete process is presented in Algorithm 1. For an earlier termination, we set the data size Top_k^Γ should return to a data range, which we verified with experiments does not ruin the backdoor defense. Another thing that should be noted is that during the protocol Top_k^Γ , the comparison result is revealed, which may reveal which sample has higher inference confidence. However, as servers do not know the concrete content and the order of the samples, the only message parties can know is that some samples have higher inference confidence than other samples, which is indistinguishable from the information obtained from a dataset with meaningless noisy samples.

Phase 2. The reversion of backdoor trigger: Owning the suspicious samples from each class, parties privately find the maximum and minimum pixel among each class. By assuming the maximum and minimum value corresponding to $x = 1$ and $x = 0$ in $y = \alpha x + (1 - \alpha)\delta$, we can get $\alpha = y_{max} - y_{min}$ and $(1 - \alpha) = y_{min}$. As we do not need to reveal the value δ , we just simplify the expression to use Δ to stand for $(1 - \alpha)\delta$. To mitigate the impact of the low backdoor sample extraction rate, CIPHERBD does not require the α to be the same value for all the pixels to approximately reverse the backdoor trigger. After reversing the trigger for each class, parties need to choose the backdoor trigger from the revealed triggers. We assume that it

Algorithm 2: Protocol of *ReTrig*.

Input: P_i inputs $\langle \mathcal{D}_b \rangle_i, \langle \mathcal{D}_s \rangle_i, \langle \theta^{ely} \rangle_i$.
Output: P_i outputs the added backdoor trigger $\langle \Delta \rangle_i$ and the backdoor trigger mask $\langle \alpha \rangle_i$.

- 1: **for** $j = 0$ to cls **do**
- 2: **for** each position (u, v) in a sample size **do**
- 3: P_i invokes $\mathcal{F}_{max}$ and $\mathcal{F}_{min}$ on $\langle y_t^{(u,v)} \rangle_i$, where $y_t \in \mathcal{D}_s^j$, gets $\langle y_{max}^{(u,v)} \rangle_i$, and $\langle y_{min}^{(u,v)} \rangle_i$ as output.
- 4: P_i sets $\langle \alpha^{(u,v)} \rangle_i := \langle y_{max}^{(u,v)} \rangle_i - \langle y_{min}^{(u,v)} \rangle_i$, $\langle \Delta^{(u,v)} \rangle_i := \langle y_{min}^{(u,v)} \rangle_i$.
- 5: **end for**
- 6: P_i sets the matrix $\langle \alpha^j \rangle_i$ with the (u, v) -position value is $\langle \alpha^{(u,v)} \rangle_i$, the same with $\langle \Delta^j \rangle_i$.
- 7: **end for**
- 8: P_i sets $poss = [l]$.
- 9: **for** $depth = 0$ to $\lceil \log cls \rceil$ **do**
- 10: **for** $j = 0$ to $\lfloor \frac{poss}{2} \rfloor$ **do**
- 11: P_i generates samples $\langle x_{test} \rangle_i := \frac{\langle \Delta^{2j} \rangle_i}{2} + \frac{\langle \Delta^{2j+1} \rangle_i}{2}$.
- 12: P_i runs model inference with $\langle \theta^{ely} \rangle_i$ and $\langle x_{test} \rangle_i$.
- 13: P_i removes the $2j$ from $poss$ if the inference result is $2j + 1$.
- 14: **end for**
- 15: **end for**
- 16: P_i set $\alpha := \alpha^\gamma, \Delta := \Delta^\gamma, l_{tgt} := \gamma$, where $\gamma := poss[0]$.

corresponds to a strongly learned feature within the model. Under this assumption, we generate the sample $x_{test} := \frac{\Delta^0}{2} + \frac{\Delta^1}{2}$ and input it into the model to determine which trigger is the stronger feature based on the inference result. The strength of each potential trigger feature is evaluated in a competitive framework, and the trigger exhibiting the strongest feature is ultimately selected as the backdoor trigger. The process is presented in Algorithm 2.

Phase 3. The cleaning of the backdoor model: After reversing the backdoor trigger and backdoor mask, CIPHERBD deploys a machine unlearning to remove the backdoor. Machine unlearning has a similar process to machine learning. The core difference is the loss function; in machine unlearning, it is not only needed to keep the task-related loss item decreasing but also to keep the task-unrelated loss item increasing. The loss function in previous works [20], [38] is $\mathcal{L} := \mu \mathcal{L}_{cln} - \beta \mathcal{L}_{poi} + \tau \mathcal{L}_\theta$. However, this form of the unlearning loss function introduces a potential instability. As the optimizer minimizes $\mathcal{L}$, the increase of $\mathcal{L}_{poi}$ may drive the overall loss to a large negative value. This can lead to excessively large parameter updates and eventually cause an over-unlearning phenomenon, even though the regularization term $\mathcal{L}_\theta$ is designed to constrain the magnitude of parameter shifts. To solve this problem, we propose the loss function $\mathcal{L} := \mu \mathcal{L}_{cln} + \frac{\beta}{\mathcal{L}_{poi} + \tau}$, where the poison loss item is posed in the denominator, which relieves the side effect caused by the large poison loss. Meanwhile, we also remove the item $\mathcal{L}_\theta$ to reduce the computation. The usefulness of the hyper-parameter

Algorithm 3: Protocol of *Punlearn*.

Input: P_i inputs $l_{tgt}, \langle \mathcal{D}_b \rangle_i, \langle \alpha \rangle_i, \langle \Delta \rangle_i, \langle \theta^{fal} \rangle_i$.
Output: P_i outputs $\langle \theta^{cln} \rangle_i$.

- 1: P_i generates dataset $\langle \mathcal{D}_g \rangle_i, \langle y \rangle_i := \langle \alpha x \rangle_i + \langle \Delta \rangle_i$, where $\langle y \rangle_i \in \langle \mathcal{D}_g \rangle_i, \langle x \rangle_i \in \langle \mathcal{D}_b \rangle_i$.
- 2: P_i defines the poison loss $\mathcal{L}_{poi} := \mathcal{L}_{CE}(F_\theta(x), l_{tgt})$ and the clean loss $\mathcal{L}_{cln} := \mathcal{L}_{CE}(F_\theta(x), l_{nat})$.
- 3: P_i conducts unlearning with the parameters θ^{fal} , by optimizing $\mathcal{L}_U := \mu \mathcal{L}_{cln} + \frac{\beta}{\mathcal{L}_{poi} + \tau}$.
- 4: P_i gets θ^{cln} after iterations of optimization.



Fig. 5. Picture revealed using only the inner comparison of max function. The left is the revealed picture, the right is the original picture.

τ is to address the cases in which the backdoor samples are well learned, causing the poison loss is approach 0 at the start of unlearning, which gives a large loss value to ruin the unlearning process. The efficiency is well supported by the experiments.

The complete protocol of CIPHERBD is described in Algorithm. 4.

C. The New Paradigm for Private Max/Min Value Extraction

In this subsection, we will present a more communication-efficient paradigm to evaluate the max/min value of a large-scale vector by utilizing the $\mathcal{F}_{Scmp}$, $\mathcal{F}_{O-Shuffle}$, which can give an efficiency improvement regardless of the concrete implementation of secure comparison. In the previous works [14], [24], the max value of an n -length array is computed iteratively by building a binary tree. In this tree, each node stores the maximum value of a sub-array until the root node stores the maximum value of the entire array. To keep the computation secrecy, it is needed to privately compute the formula $\langle z \rangle_i = \langle x \rangle_i + \langle y - x \rangle_i \cdot \langle \{y > x\} \rangle_i$ $n - 1$ times. In each time of computation, a Bit2A conversion from $\mathbb{Z}_2$ to $\mathbb{Z}_{2^l}$ is needed to ensure the $\langle y - x \rangle_i \cdot \langle \{y > x\} \rangle_i$ computed in the $\mathbb{Z}_{2^l}$, which incurs a 2 rounds online communication with the size of $2 + 4l$. Thus, the max operation of n elements array will have additional $2 \log n$ rounds and $(n - 1)(2 + 4l)$ size of communication except for the comparison operation. To eliminate this portion of communication overhead, we consider revealing the comparison results between pixels from different samples. However, directly revealing these results may leak structural information about the image, as illustrated in Fig. 5. Although absolute pixel values remain hidden, the comparison outcomes preserve the relative ordering among pixels. Due to the strong spatial correlation and

Algorithm 4: Protocol of CIPHERBD.

Input: P_i inputs $\langle \mathcal{D} \rangle_i, \langle \mathcal{G} \rangle_i, \langle \theta^{ely} \rangle_i, \langle \theta^{fal} \rangle_i, k, \Gamma$.
Output: P_i outputs θ_i^{cln} .

- 1: **for** $j = 0$ to cls **do**
- 2: P_i computes the gini coefficient $\langle gin_t \rangle_i := \langle \vec{p}_t \rangle_i$, where $\langle \vec{p}_t \rangle_i \in \langle \mathcal{G}^j \rangle_i$.
- 3: P_i invokes Top_k^Γ and Bot_k^Γ with inputs $\{\langle gin_1 \rangle_i, \dots, \langle gin_n \rangle_i\}, k, \Gamma$, gets the indices idx_T^j and idx_B^j .
- 4:4: **end for**
- 5: P_i selects $\langle \mathcal{D}_b \rangle_i, \langle \mathcal{D}_s \rangle_i$ with idx_B and idx_T from $\langle \mathcal{D} \rangle_i$.
- 6: P_i invokes $ReTrig$ with inputs $\langle \mathcal{D}_b \rangle_i, \langle \mathcal{D}_s \rangle_i, \langle \theta^{ely} \rangle_i$, gets $\langle \alpha \rangle_i$ and $\langle \Delta \rangle_i$.
- 7: P_i invokes $Punlearn$ with inputs $\langle \mathcal{D}_b \rangle_i, \langle \alpha \rangle_i, \langle \Delta \rangle_i, \langle \theta^{fal} \rangle_i$, gets $\langle \theta^{cln} \rangle_i$.

Algorithm 5: Protocol of *ShufMax*.

Input: P_i inputs an array $\langle X \rangle_i := \{\langle x_0 \rangle_i, \dots, \langle x_{n-1} \rangle_i\}$.
Output: P_i outputs $\langle x_{max} \rangle_i$.

- 1: P_i invokes $\mathcal{F}_{O-Shuffle}$ with input $\langle X \rangle_i$, gets the shuffled array $\langle arr \rangle_i$ as output.
- 2: **for** $t = 1$ to $\lceil \log n \rceil$ **do**
- 3: **for** $j = 0$ to $\lfloor \frac{\lfloor \langle arr \rangle_i \rfloor}{2} - 1 \rfloor$ **do**
- 4: P_i computes $\langle b \rangle_i := \mathcal{F}_{SCmp}(\langle arr_{2j} \rangle_i, \langle arr_{2j+1} \rangle_i)$, and reveals b .
- 5: P_i adds $\langle arr_{2j} \rangle_i$ to $\langle Nxtarr \rangle_i$ if $b = 1$ else adds $\langle arr_{2j+1} \rangle_i$ to $\langle Nxtarr \rangle_i$.
- 6: **end for**
- 7: **if** $\lfloor \langle arr \rangle_i \rfloor$ is odd **then**
- 8: P_i adds $\langle arr_{\lfloor \langle arr \rangle_i \rfloor} \rangle_i$ to $\langle Nxtarr \rangle_i$.
- 9: **end if**
- 10: P_i sets $\langle arr \rangle_i := \langle Nxtarr \rangle_i$.
- 11: **end for**
- 12: P_i sets $\langle x_{max} \rangle_i := \langle arr_0 \rangle_i$.

local smoothness of natural images, neighboring pixels typically have stable local ordering relations. Consequently, the collection of comparison results implicitly preserves the rank structure of the image, which reveals coarse spatial patterns such as edges and regions.

To mitigate this leakage, the proposed *ShufMax* protocol introduces an oblivious shuffle in the pixel dimension before revealing any comparison outcomes. Specifically, pixel positions are randomly permuted so that the revealed comparison results are no longer associated with their original spatial locations. As a result, even though comparison outcomes are revealed, the adversary only observes comparisons between randomly permuted pixel indices. This permutation breaks the spatial correlation between neighboring pixels, thereby preventing the recovery of image structures. Through oblivious shuffle, a max operation on a n elements array can save the overhead of Bit2A conversion and the private multiplication, which in total $(n-1)(2+4l)$ bits communication. The additional oblivious shuffle [25] operation comes at the cost of $2nl$ and the cost of reversing the comparison result, which costs $2(n-1)$ bits. Such that by shuffling the array first, we can get a $2(n-2)l$ bits per max(min) operation and also $(n-1)$ rounds communication to perform private multiplication. The complete process of protocol *ShufMax* is described in Algorithm 5.

D. Security Proof of *ShufMax*

In this subsection, we give the security proof of *ShufMax*.

Theorem 1: The protocol *ShufMax* as shown in Algorithm 5 UC realizes $\mathcal{F}_{max}$ in the $(\mathcal{F}_{O-Shuffle}, \mathcal{F}_{SCmp})$ -hybrid model against semi-honest probabilistic polynomial time (PPT) adversaries with static corruption.

Proof: To prove Theorem 1, we construct a PPT simulator $\mathcal{S}$, such that no non-uniform PPT environment $\mathcal{Z}$ can distinguish between the ideal world $\text{Ideal}_{\mathcal{F}_{max}, \mathcal{S}, \mathcal{Z}}(1^\lambda)$ and the real world $\text{Real}_{\text{ShufMax}, \mathcal{S}, \mathcal{Z}}(1^\lambda)$. We consider the following cases:

Case 1: P_0 is corrupted. We construct the simulator $\mathcal{S}$ that internally runs $\mathcal{A}$, simulates $\mathcal{F}_{SCmp}$ and $\mathcal{F}_{O-Shuffle}$, forwards messages to/from $\mathcal{Z}$, and simulates the interface of the honest party P_1 .

Upon receiving $(\text{Max}, \text{sid}, P_1)$ from $\mathcal{F}_{max}$, $\mathcal{S}$ does as follows.

- $\mathcal{S}$ simulates the input of P_1 $\langle \vec{x} \rangle_1 \xleftarrow{\$} \mathbb{Z}_{2^n}^n$.
- Upon $\mathcal{S}$ receiving $(\text{O-Shuffle}, \text{sid}, \langle \vec{x} \rangle_0)$ from P_0 and receiving $(\text{Return}, \text{sid}, \langle y \rangle_0)$ from $\mathcal{F}_{max}$, $\mathcal{S}$ simulates $\mathcal{F}_{O-Shuffle}$ to sends $\vec{y}[\text{idx}] = \langle y \rangle_0$, where $\text{idx} \xleftarrow{\$} [n]$ and the other places are random numbers in $\mathbb{Z}_{2^n}$ to P_0 .
- Upon $\mathcal{S}$ receiving $(\text{SCmp}, \text{sid}, \langle x \rangle_0, \langle y \rangle_0)$ from P_0 , $\mathcal{S}$ simulates $\mathcal{F}_{SCmp}$ to sends $\text{flag}_0 \xleftarrow{\$} \mathbb{Z}_2$ to P_0 as $\langle \text{flag} \rangle_0$ and sends flag_1 , such that $\text{flag}_0 + \text{flag}_1$ ensures the value in the idx -th position will be selected into next level, which is communicated to P_0 .

Indistinguishability. We show that the incoming message and the output of P_0 in the ideal world are indistinguishable from the real world.

Claim 1: The ideal world $\text{Ideal}_{\mathcal{F}_{max}, \mathcal{S}, \mathcal{Z}}(1^\lambda)$ and the real world $\text{Real}_{\text{ShufMax}, \mathcal{S}, \mathcal{Z}}(1^\lambda)$ are perfectly indistinguishable.

There are three distinct components of the incoming messages that differ between $\text{Ideal}_{\mathcal{F}_{max}, \mathcal{S}, \mathcal{Z}}(1^\lambda)$ and the real world $\text{Real}_{\text{ShufMax}, \mathcal{S}, \mathcal{Z}}(1^\lambda)$.

- The output of $\mathcal{F}_{O-Shuffle}$ to P_0 is a vector of random numbers indistinguishable between the ideal world and the real world. This is ensured by the functionality of $\mathcal{F}_{O-Shuffle}$.
- The output of $\mathcal{F}_{SCmp}$ to P_0 is a random bit flag_0 , indistinguishable between the ideal world and the real world. This is ensured by the functionality of $\mathcal{F}_{SCmp}$.
- The bit flag_1 is indistinguishable between the ideal world and the real world according to claim 2.

Claim 2: flag_1 the ideal world and $\langle \text{flag} \rangle_1$ in the real world are perfectly indistinguishable.

Proof: For any two numbers m, n in $\vec{x}$, then according to the functionality $\mathcal{F}_{\text{O-Shuffle}}$, we can get

$$P(\vec{x}[0] = m, \vec{x}[1] = n) = P(\vec{x}[0] = n, \vec{x}[1] = m)$$

such that

$$P(\vec{x}[0] > \vec{x}[1]) = P(\vec{x}[1] > \vec{x}[0])$$

$$P(flag=1) = \frac{1}{2}$$

$$P(\langle flag \rangle_0 = 1) = \frac{1}{2}$$

$$P(\langle flag \rangle_1 = 1) = P(flag=0 | \langle flag \rangle_0 = 1) \cdot P(\langle flag \rangle_0 = 1) \\ + P(flag=1 | \langle flag \rangle_0 = 0) \cdot P(\langle flag \rangle_0 = 0)$$

$$P(\langle flag \rangle_1 = 1) = P(flag=0) \cdot P(\langle flag \rangle_0 = 1) \\ + P(flag=1) \cdot P(\langle flag \rangle_0 = 0)$$

$$P(\langle flag \rangle_1 = 1) = \frac{1}{2} = P(flag_1)$$

□
□

The scenario in which P_1 is corrupted is analogous to the case where P_0 is corrupted; therefore, we omit the discussion of this case.

This concludes the proof.

VI. EXPERIMENTS

In this section, we present comprehensive experiments conducted to study the effectiveness of our backdoor defense CIPHERBD and the efficiency improvement of our *Max* protocol.

A. Experimental Setup

Implementation: Our experiment code is based on the crypten framework [14]. Following most PPNN settings, we employ a fixed-point secret sharing scheme with a representation of 64 bits, out of which 16 bits are allocated for the fractional portion. Our experimental evaluations are conducted on systems featuring an Intel(R) Xeon(R) Silver 4116 CPU @ 2.10 GHz with 128 GB of RAM.

Tested networks, datasets, and backdoor attacks: To show the fitness of CIPHERBD in the MPC context, we set the backdoor defense Reback [20] as a baseline. To get a more general result, we test CIPHERBD in three widely recognized public benchmark datasets, namely SVHN [35], CIFAR-10 [40], and GTSRB [41], and choose three mainly used model architecture, namely 5-Layer CNN, LeNet [42], vgg16 [43], within three popular backdoor attack badnet [26], blend [27], trojan [28]. A comprehensive overview of the three datasets is provided below:

- *SVHN:* The SVHN dataset consists of 73257 labeled images of house numbers from Google Street View, spanning 10 classes (digits 0-9).

TABLE III
THE RUNTIME ESTIMATION OF DIFFERENT BACKDOOR DEFENSES ON CIFAR-10 DATASET. EACH TABLE ENTRY RECORDS THE ASSOCIATED NUMBER OF ITERATIONS AND THE BATCH SIZE

Schemes	Train iter $\times 10^2$	Infer iter $\times 10^2$	runtime (day)
DBD	7820.00 (128)	-	1251.48
ASD	1228.80 (256)	245.76 (64)	410.21
PSBD	391.00 (128)	463.52 (128)	136.50
CT	1631.20 (64)	280.00 (32)	133.94
TSC	122.10 (256)	74.18 (256)	53.81
NC	189.00 (32)	15.64 (128)	10.86
Spectre	39.10 (128)	-	6.25
SCAn	39.10 (128)	-	6.25
Reback	0.64 (128)	0.20 (256)	0.15
CIPHERBD	0.64 (128)	0.09 (1)	0.10

- *CIFAR-10:* The CIFAR-10 dataset consists of 60000 images across 10 classes, representing everyday objects like airplanes, cars, and animals. The dataset is divided into training (50000) and testing (10000) subsets.
- *GTSRB:* The GTSRB dataset contains over 50000 images of traffic signs across 43 classes, used for training and testing traffic sign recognition models.

B. The Cost of Existing Backdoor Defense in MPC Context

In this subsection, we evaluate the computational cost of various backdoor defense mechanisms, focusing on the iteration of model training and model inference, which is the most costly part of each defense. Due to the impractical cost caused by the MPC operation and the large iterations for training and inference, we estimate the number of iterations based on the corresponding batch sizes and use this to approximate the overall runtime.

As shown in Table III, DBD, ASD, and CT, which leverage the mechanism to inhibit the backdoor learning, require approximately 40 to 200 times more iterations as compared to the standard learning process, resulting in runtime exceeding four months. Similar to CIPHERBD, NC also mitigates the backdoor through an unlearning approach; however, its trigger reversion relies on iterative optimization, requiring an additional 18900 iterations and taking over 10 days to complete. Spectre and SCAn employ class-wise statistical analysis to identify backdoor samples and retrain a clean model on a sanitized dataset, which doubles the training cost and takes more than six days to yield a clean model. This cost analysis reveals that most existing backdoor defense mechanisms significantly increase training iterations compared to standard model training, making them impractical for deployment in the MPC context. In contrast, Reback and CIPHERBD implement unlearning schemes that reverse the backdoor trigger using the mathematical structure of backdoor samples rather than relying on gradient optimization. This makes both approaches more practical for MPC settings.

C. Effectiveness of CIPHERBD

Even if Reback exhibits low complexity suitable for use in the MPC context, it remains ineffective in mitigating backdoors due to its reliance on high computational precision. In this subsection, we conduct experiments to evaluate the effectiveness of both Reback and CIPHERBD. Compared to Reback, CIPHERBD

TABLE IV
BACKDOOR SAMPLES EXTRACTION RATE WITH ENTROPY AND GINI
COEFFICIENT UNDER BADNET, BLEND, TROJAN ATTACKS AND CNN, LENET,
VGG16 NETWORKS USING THE SVHN DATASET

Networks & Attacks		entropy plain	entropy MPC	CIPHERBD
cnn	badnet	100%	60%	100%
	blend	100%	12%	88%
	trojan	100%	28%	94%
lenet	badnet	90%	90%	100%
	blend	94%	90%	86%
	trojan	100%	40%	100%
vgg16	badnet	100%	12%	98%
	blend	98%	70%	100%
	trojan	100%	82%	96%

introduces three key modifications to enhance MPC adaptability: (1) a revised standard for high inference confidence, (2) a relaxed constraint on the backdoor mask α , and (3) an updated unlearning loss function. We present experimental results for both CIPHERBD and Reback to highlight their differences in effectiveness.

The backdoor samples extraction rate: In Reback, the backdoor samples are extracted based on the model inference, and the model inference of a sample is tested by the entropy $H(X) = -\sum_{i=1}^n p(x_i) \log p(x_i)$. While in datasets with prominent features, such as SVHN, certain benign samples are easily learned during the early training epochs, making the distinction between backdoor and benign samples less apparent. Moreover, in the context of MPC, the logarithmic function is approximated using Householder iterations [37], introducing computational deviations under low-precision arithmetic—particularly when the input approaches 1. This scenario often corresponds to samples with high inference confidence and leads to a significant decrease in the backdoor sample extraction rate. As shown in Table IV, in the plaintext context, the entropy used as the metric for inference confidence results in a high backdoor extraction rate, approaching nearly 100%. In contrast, within the context of MPC, the backdoor extraction rate significantly decreases, showing a reduction ranging from 18% to 88%. To address the approximation errors in logarithmic computation, the Gini coefficient can be employed as an alternative metric in the MPC setting, resulting in improved backdoor extraction rates compared to the entropy-based method. Nevertheless, the extraction rate under MPC still remains lower than that achieved in the plaintext setting.

The trigger recovery: Although the Gini coefficient improves the extraction rate of backdoor samples, it remains ineffective in accurately reconstructing the backdoor trigger. Here, we present the backdoor triggers reconstructed by Reback and CIPHERBD using samples selected based on the Gini coefficient, highlighting the robustness of our pixel-independent trigger approximation.

As illustrated in Fig. 6, CIPHERBD effectively recovers the most salient components of the backdoor trigger, albeit with some noise when compared to the original trigger. In certain cases, the trigger reversed by Reback is different from the original trigger. This phenomenon arises because Reback assumes a constant value for α across all pixels in the expression $y = \alpha x + \Delta$. When the number of extracted backdoor samples is small, the pixel values of benign samples may exceed those of



Fig. 6. The trigger reversed by Reback(Top) and CIPHERBD (Bottom) against blend backdoor attack on cnn, lenet, and vgg16 networks with the suspicious samples extracted with the Gini coefficient.

the backdoor samples. Consequently, the value of α computed for these pixels can be larger than the original α chosen by the attacker. This inaccurate estimation propagates throughout the entire image, as Reback applies a uniform α across all pixel positions, ultimately leading to a suboptimal recovery of the backdoor trigger. In contrast, CIPHERBD assigns a distinct α to each pixel position, thereby localizing the effect of inaccuracies in α to individual pixels. This strategy enhances the robustness of the backdoor trigger recovery process, as it minimizes the impact of errors on the overall image reconstruction.

The effectiveness of model cleaning: To evaluate the effectiveness of our approximate reversed backdoor trigger, we conduct machine unlearning experiments using both the trigger reversed by CIPHERBD and the original backdoor trigger. As illustrated in Table V, the attack success rate (ASR) across all datasets and network architectures decreases by more than 90%, while the accuracy (ACC) remains unchanged after applying CIPHERBD. Moreover, the ASR achieved using CIPHERBD is comparable to that obtained using the original trigger, with an average deviation of less than 1%. These results demonstrate that the reversed backdoor trigger generated by CIPHERBD is as effective as the original in enabling backdoor unlearning without compromising model performance. Additionally, the model's accuracy remains stable, indicating that the reversed trigger does not cause significant forgetting of features from benign samples.

To evaluate the effectiveness of the loss function in CIPHERBD, we record the unlearning curves for both the loss function used in Reback and the one implemented in CIPHERBD, with the trigger reversed by CIPHERBD. The primary difference between these two loss functions lies in the placement of the backdoor loss term, $\mathcal{L}_{poi}$. In backdoor attacks, the backdoor samples should have a minimal loss, as they are well-learned by the model. Therefore, in the loss function $\mathcal{L} := \mu\mathcal{L}_{cln} - \beta\mathcal{L}_{poi} + \tau\mathcal{L}_\theta$, $\mathcal{L}_{poi}$ contributes only marginally to the unlearning of the backdoor. Furthermore, after multiple epochs of unlearning, $\mathcal{L}$ is artificially assigned a large negative value due to the subtraction of a large $\mathcal{L}_{poi}$, which detrimentally affects the accuracy of benign samples.

As shown in Fig. 7, we observe that, compared to the loss function in Reback, which results in a delayed decrease in ASR and over-forgetting of benign samples, the loss function in CIPHERBD demonstrates improved performance. Specifically, the loss function in CIPHERBD, $\mathcal{L} := \mu\mathcal{L}_{cln} + \frac{\beta}{\mathcal{L}_{poi} + \tau}$, places

TABLE V
THE UNLEARNING EFFECTIVENESS WITH THE ORIGINAL TRIGGER AND THE TRIGGER REVERSED BY CIPHERBD

Backdoor Attack	Models	CIFAR-10						SVHN						GTSRB					
		Poisoned		Ori-clean		CIPHERBD		Poisoned		Ori-clean		CIPHERBD		Poisoned		Ori-clean		CIPHERBD	
		ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC
badnet	cnn	99.61	73.64	0.29	71.63	0.24	71.58	99.55	89.71	6.64	90.44	7.96	89.97	98.74	78.98	0.05	92.91	0.03	93.25
	lenet	99.24	74.52	1.61	69.68	1.50	70.01	98.43	89.92	13.28	90.27	5.59	90.24	97.90	86.89	25.69	95.83	7.20	95.83
	vgg16	99.87	72.06	1.88	73.18	1.24	72.80	99.48	92.34	0.93	93.65	0.61	93.70	99.92	94.57	0.01	97.79	0.01	97.74
blend	cnn	99.62	70.84	0.06	70.04	0.03	69.81	99.18	88.82	0.02	88.18	0.21	89.64	99.00	83.00	0.01	94.51	0.09	94.89
	lenet	99.85	73.94	0.01	69.33	0.06	69.18	99.19	85.42	0.01	85.04	0.42	86.12	99.05	85.31	0.23	95.53	1.11	95.40
	vgg16	99.75	76.74	0.07	75.68	0.11	75.49	99.54	91.43	0.07	92.72	0.12	92.44	99.74	92.64	0.41	97.17	0.00	97.18
trojan	cnn	99.32	72.44	0.22	71.19	0.11	70.88	97.53	89.02	0.12	90.26	0.24	90.67	99.68	83.94	0.27	94.85	0.21	95.09
	lenet	99.36	74.06	1.17	70.11	0.54	70.10	99.81	89.63	0.98	89.84	0.60	90.33	92.48	77.77	1.64	93.82	0.75	93.73
	vgg16	99.90	75.88	0.31	74.34	0.01	73.98	99.91	92.87	0.40	90.61	1.28	93.46	99.96	91.65	0.00	96.84	0.00	96.98

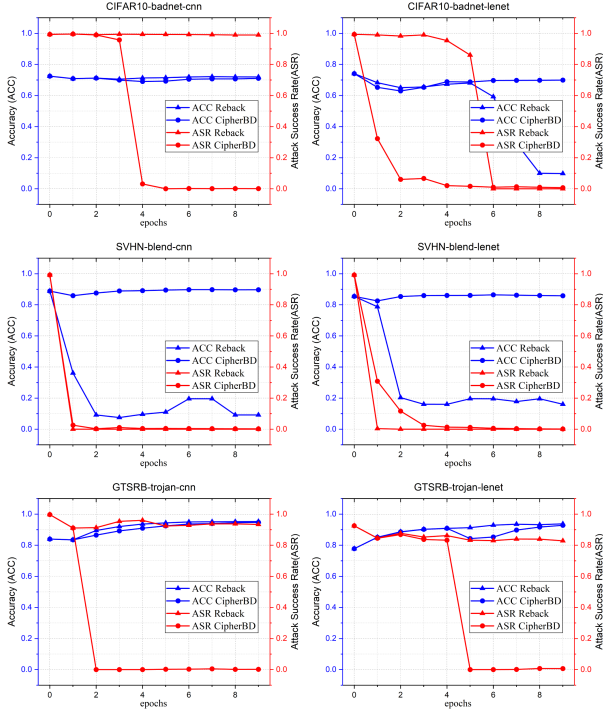


Fig. 7. The learning curve of backdoor unlearning with the original trigger and reversed trigger.

$\mathcal{L}_{poi}$ in the denominator. This ensures that the overall loss $\mathcal{L}$ does not become extremely large, preventing over-unlearning. Furthermore, the addition of τ in the denominator avoids excessively large loss values during the early epochs, when $\mathcal{L}_{poi}$ is small. Also, as shown in Fig. 8, the learning curve of backdoor unlearning with the parameter regularization term $\mathcal{L}_{\theta}$ is almost identical to that of CIPHERBD. The Pearson correlation coefficient between the two loss curves is nearly 1, indicating that they are highly consistent. This result suggests that $\mathcal{L}_{\theta}$ contributes little to preventing the over-unlearning phenomenon.

D. Overall Cost of CIPHERBD

Although Reback does not require iterative optimization, it still faces performance bottlenecks, particularly when handling a large number of label classes. As the defender lacks knowledge about the backdoor target selected by the adversary, there are l potential triggers corresponding to each class. To confirm the backdoor trigger, Reback leverages the characteristic that the backdoor trigger can transform samples from other classes

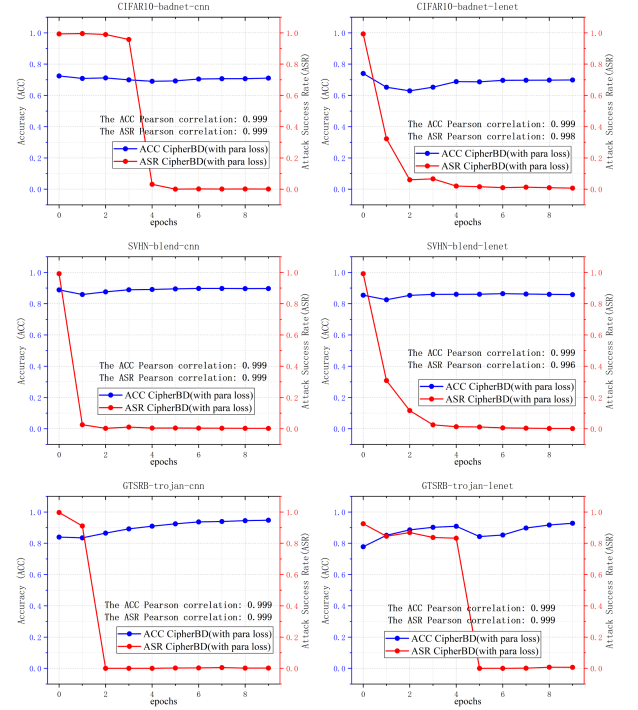


Fig. 8. Learning curves of the unlearning process with and without parameter regularization, along with their Pearson correlation coefficient.

into the backdoor target. Based on this intuition, Reback needs to test about ζl^2 samples with the model inference, leading to the cost growing quadratically with the number of classes. To mitigate the cost associated with identifying backdoor triggers, CIPHERBD introduces a feature strength test based on the assumption that the backdoor trigger represents the most prominent feature, which only requires to test $l - 1$ samples. CIPHERBD reduces the number of private model inferences by a factor of $500\times$ compared to Reback, resulting in a substantial improvement in efficiency. As shown in Table VI and Table VII, confirming the backdoor target from suspicious labels is the most expensive component in Reback. By applying our feature strength test, the cost of this step is substantially reduced.

E. Efficiency Improvement of the New Paradigm of Max/Min

As a fundamental component of our trigger approximation method, *ShufMax* leverages the oblivious shuffle to hide the relation of elements in the array, reducing the cost to privately

TABLE VI
THE RUNTIME OF EACH PART IN REBACK AND CIPHERBD ON GTSRB
DATASET WITH 43 CLASSES

Schemes (s)		suspicious extraction	trigger reveal	suspicious label	unlearning
cnn	Reback	683.69	125.67	61310.99	2878.52
	CIPHERBD	683.69	125.67	27.85	2878.52
lenet	Reback	755.11	125.67	73347.98	5669.95
	CIPHERBD	755.11	125.67	33.32	5669.95
vgg16	Reback	744.64	125.67	205322.21	29362.75
	CIPHERBD	744.64	125.67	93.28	29362.75

TABLE VII
THE COMMUNICATION OF EACH PART IN REBACK AND CIPHERBD ON GTSRB
DATASET WITH 43 CLASSES

Schemes (GB)		suspicious extraction	trigger reveal	suspicious label	unlearning
cnn	Reback	4.35	18.46	9580.17	415.95
	CIPHERBD	4.35	18.46	4.35	415.95
lenet	Reback	4.99	18.46	10986.38	769.58
	CIPHERBD	4.99	18.46	4.99	769.58
vgg16	Reback	12.40	18.46	27301.26	2164.49
	CIPHERBD	12.40	18.46	12.40	2164.49

TABLE VIII
THE RUNTIME AND COMMUNICATION ON TRADITIONAL PARADIGM AND
ShufMax

Array Length	Methods	LAN(ms)	WAN(ms)	Comm.(KB)
16	Traditional	21	2809	126.16
	<i>ShufMax</i>	19	2511	70.45
32	Traditional	34	4519	238.38
	<i>ShufMax</i>	23	3015	126.97
64	Traditional	61	8032	456.29
	<i>ShufMax</i>	25	3518	236.74
128	Traditional	108	15157	885.55
	<i>ShufMax</i>	35	4022	453.01

choose the larger element. To demonstrate the efficiency improvement of *ShufMax*, we compared it with the traditional maximum finding paradigm and the secure comparison method proposed in [33]. To simulate various network conditions, we used Linux Traffic Control (tc), emulating a LAN environment with 10 Gbps bandwidth and 0.125 ms round-trip (RTT) latency and a WAN environment with 1 Gbps bandwidth and 50 ms RTT latency. The performance was evaluated for array sizes ranging from 16 to 128, with each element having a size of 64 bits. We test 5000 instances under both LAN and WAN settings, and record the cost per instance in Table VIII, by utilizing an oblivious shuffle to permute the array before the comparison, *ShufMax* enables the reveal of the comparison result. This approach reduces communication cost by about 44% to 48%, and about 9% to 67% runtime under LAN settings, and about 10% to 73% runtime under WAN conditions.

VII. CONCLUSION

In this work, we present CIPHERBD, a tailored backdoor defense framework for privacy-preserving neural network training. By adapting the efficient Reback defense to MPC settings, CIPHERBD addresses the unique challenges posed by

MPC, including computational overhead and limited data representation. Through key optimizations, including a reduction in private model inferences and an improved backdoor sample extraction method, CIPHERBD significantly enhances defense efficiency. Additionally, its flexible trigger recovery approach and adaptive unlearning loss function further improve backdoor mitigation robustness under precision constraints. Extensive evaluations across multiple neural network architectures and benchmark datasets confirm the effectiveness of CIPHERBD against leading backdoor attack strategies, marking a significant advancement in PPNN training.

REFERENCES

- [1] N. Attrapadung et al., "Adam in private: Secure and fast training of deep neural networks with adaptive moment estimation," in *Proc. Privacy Enhancing Technol.*, vol. 4, 2022, pp. 746–767.
- [2] H. Chaudhari, R. Rachuri, and A. Suresh, "Trident: Efficient 4 PC framework for privacy preserving machine learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020.
- [3] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "{ABY2. 0}: Improved {mixed-protocol} secure {two-party} computation," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2165–2182.
- [4] S. Tan, B. Knott, Y. Tian, and D. J. Wu, "CryptGPU: Fast privacy-preserving machine learning on the GPU," in *Proc. IEEE Symp. Secur. Privacy*, 2021, pp. 1021–1038.
- [5] J.-L. Watson, S. Wagh, and R. A. Popa, "Piranha: A {GPU} platform for secure computation," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 827–844.
- [6] Y. Liu, W.-C. Lee, G. Tao, S. Ma, Y. Aafer, and X. Zhang, "ABS: Scanning neural networks for back-doors by artificial brain stimulation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1265–1282.
- [7] B. Wang et al., "Neural cleanse: Identifying and mitigating backdoor attacks in neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 707–723.
- [8] K. Gao, Y. Bai, J. Gu, Y. Yang, and S.-T. Xia, "Backdoor defense via adaptively splitting poisoned dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 4005–4014.
- [9] X. Qi, T. Xie, J. T. Wang, T. Wu, S. Mahloujifar, and P. Mittal, "Towards a proactive {ML} approach for detecting backdoor poison samples," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 1685–1702.
- [10] K. Huang, Y. Li, B. Wu, Z. Qin, and K. Ren, "Backdoor defense via decoupling the training process," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [11] G. Shen et al., "Backdoor scanning for deep neural networks through K-arm optimization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 9525–9536.
- [12] J. Hayase, W. Kong, R. Somani, and S. Oh, "SPECTRE: Defending against backdoor attacks using robust statistics," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 4129–4139.
- [13] D. Tang, X. Wang, H. Tang, and K. Zhang, "Demon in the variant: Statistical analysis of {DNNs} for robust backdoor contamination detection," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1541–1558.
- [14] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten, "CRYPTEN: Secure multi-party computation meets machine learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 4961–4973.
- [15] A. R. W. A. Khan and M. A. Noor, "Higher-order iterative methods by using householders method for solving certain nonlinear equations," *Math. Sci. Lett.*, vol. 2, no. 2, pp. 107–120, 2013.
- [16] W. Li, P.-Y. Chen, S. Liu, and R. Wang, "PSBD: Prediction shift uncertainty unlocks backdoor detection," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2025, pp. 10255–10264.
- [17] Z. Yan, S. Li, R. Zhao, Y. Tian, and Y. Zhao, "DHBE: Data-free holistic backdoor erasing in deep neural networks via restricted adversarial distillation," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2023, pp. 731–745.
- [18] J. Peng et al., "Circumventing backdoor space via weight symmetry," in *Proc. Int. Conf. Mach. Learn. PMLR*, 2025, pp. 48886–48922.
- [19] Z. Yan, J. Wu, G. Li, S. Li, and M. Guizani, "Deep neural backdoor in semi-supervised learning: Threats and countermeasures," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4827–4842, 2021.

- [20] Z. Ma et al., "Need for speed: Taming backdoor attacks with speed and precision," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 1217–1235.
- [21] E. Rose, H. Lycklama, H. Chaudhari, A. Hithnawi, and A. Oprea, "UTrace: Poisoning forensics for private collaborative learning," 2024, *arXiv:2409.15126*.
- [22] H. Lycklama, A. Viand, N. K  chler, C. Knabenhans, and A. Hithnawi, "Holding secrets accountable: Auditing privacy-preserving machine learning," in *Proc. USENIX Secur. Symp.*, 2024.
- [23] D. Beaver, "Efficient multiparty protocols using circuit randomization," in *Proc. Adv. Cryptol. Proc.*, 1992, pp. 420–432.
- [24] D. Rathee et al., "CrypTFlow2: Practical 2-party secure inference," in *Proc. 2020 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 325–342.
- [25] M. Chase, E. Ghosh, and O. Poburinnaya, "Secret-shared shuffle," in *Proc. Adv. Cryptol. 26th Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Daejeon, South Korea, 2020, pp. 342–372.
- [26] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying vulnerabilities in the machine learning model supply chain," in *Proc. Mach. Learn. Comput. Secur. (MLSec) NIPS Workshop*, 2017.
- [27] X. Chen, C. Liu, B. Li, K. Lu, and D. Song, "Targeted backdoor attacks on deep learning systems using data poisoning," 2017, *arXiv:1712.05526*.
- [28] Y. Liu et al., "Trojaning attack on neural networks," in *Proc. 25th Annu. Netw. Distrib. System Secur. Symp.*, 2018, pp. 1–15.
- [29] M. Barni, K. Kallas, and B. Tondi, "A new backdoor attack in CNNs by training set corruption without label poisoning," in *Proc. IEEE Int. Conf. Image Process.*, 2019, pp. 101–105.
- [30] Y. Yang, Z. Ma, Y. Li, Y. Liu, X. Liu, and J. Ma, "S-Teapot: Swift and efficient defense against patch-based backdoor attack," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 4176–4188, Jul./Aug. 2025.
- [31] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. 42nd IEEE Symp. Foundations Comput. Sci.*, 2001, pp. 136–145.
- [32] W.-j. Lu et al., "Bumblebee: Secure two-party inference framework for large transformers," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2025, pp. 1–18.
- [33] T. Lu et al., "Efficient 2 PC for constant round secure equality testing and comparison," in *Proc. 34th USENIX Secur. Symp. (USENIX Security)*, 2025, pp. 1985–2004.
- [34] T. Wang, Y. Yao, F. Xu, M. Xu, S. An, and T. Wang, "Inspecting prediction confidence for detecting black-box backdoor attacks," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 1, pp. 274–282.
- [35] Y. Netzer et al., "Reading digits in natural images with unsupervised feature learning," in *Proc. NIPS Workshop Deep Learn. Unsupervised Feature Learn.*, vol. 2011, no. 2, 2011, pp. 1–9.
- [36] D. Rathee et al., "SiRnn: A math library for secure RNN inference," in *Proc. IEEE Symp. Secur. Privacy*, 2021, pp. 1003–1020.
- [37] A. S. Householder, "The numerical treatment of a single nonlinear equation," 1970.
- [38] Y. Liu et al., "Backdoor defense with machine unlearning," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 280–289.
- [39] C. A. Hoare, "Algorithm 65: Find," *Commun. ACM*, vol. 4, no. 7, pp. 321–322, 1961.
- [40] K. Alex, "Learning multiple layers of features from tiny images," 2009. [Online]. Available: <https://www.cs.toronto.edu/kriz/learning-features-2009-TR.pdf>
- [41] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "The German traffic sign recognition benchmark: A multi-class classification competition," in *Proc. Int. Joint Conf. Neural Netw.*, 2011, pp. 1453–1460.
- [42] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [43] K. Simonyan, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.



Tanren Liu received the BS degree in cybersecurity from Xidian University, Xi'an, China, in 2023. He is currently working toward the postgraduation degree with the School of Cyber Engineering, Xidian University. His research focuses on data security and privacy protection.



Zhuzhu Wang received the PhD degree in cyber security from Xidian University, Xi'an, China, in 2023. She is currently an associate professor with the School of Information Science and Technology, Northwest University. Her research interests include secret sharing, secure computation, and deep learning neural networks in cyber security.



Xin Kang received the MS degree in cybersecurity from Xidian University, Xi'an, China, in 2022. She is currently working toward the doctoral degree with the School of Cyber Engineering, Xidian University. Her research focuses on data security and privacy protection.



Yang Liu received the PhD degree in cyber security from Xidian University, Xi'an, China, in 2022. He is currently an associate professor with the School of Cyber Engineering, Xidian University. His research interests include privacy-preserving computation and AI security.



Yilong Yang received the BS degree in network engineering from Xidian University, in 2019, where he is currently working toward the master's degree with the School of Cyber Engineering. His research interests include analysis of protocols, deep learning neural networks, and network measurement.



Zhihong Liu (Member, IEEE) received the PhD degree in cryptography from Xidian University, in 2009. He is currently an associate professor with the School of Cyber Engineering, Xidian University. His research interests include wide range of areas including cryptography, distributed computing, and wireless physical layer security.



Zhuo Ma (Senior Member, IEEE) received the PhD degree in computer architecture from Xidian University, Xi'an, China, in 2010. He is currently a professor with the School of Cyber Engineering, Xidian University. His research interests include cryptography, AI security, and IoT security. He awards "Huashan Scholars" Distinguished Professor.